

CH10 — Pointers with Functions

SLIDE 1 — Why Use Pointers with Functions?

Normally, when we pass a variable to a function, the function gets a **copy** of its value.

Example:

```
None
void change(int x)
{
    x = 50;
}
```

Changing `x` inside the function does **not** change the original variable.

With a pointer, we can pass the **address** of the variable.

```
None
void change(int *p)
{
    *p = 50;
}
```

Easy Analogy:

Imagine giving someone the **address of your house** instead of giving them a photo of your house.

With the address, they can reach and change the original thing.

Remember:

Normal variable → COPY

Pointer → ADDRESS → ORIGINAL VALUE

SLIDE 2 — Passing an Address to a Function

Suppose:

None

```
int x = 10;
```

We can send its address to a function:

None

```
change(&x);
```

The function receives it using a pointer:

None

```
void change(int *p)
{
    *p = 20;
}
```

 **Look at this:**

None

```
x = 10
  ↓
&x
  ↓
p
  ↓
*p = 20
  ↓
x = 20
```

The pointer allows the function to access the **original variable**.

 **Memory Trick:**

&x → SEND ADDRESS

int *p → RECEIVE ADDRESS

***p → ACCESS VALUE**

SLIDE 3 — Changing a Variable Inside a Function

Example:

```
None
#include <stdio.h>

void change(int *p)
{
    *p = 100;
}

int main()
{
    int x = 10;

    change(&x);

    printf("%d", x);

    return 0;
}
```

Output:

```
None
100
```

 **Why?**

Before function:

```
None
x = 10
```

Function receives:

None

`&x`

↓

`p`

Then:

None

`*p = 100;`

changes the **original x**.

So:

None

`x = 100`

★ **Remember:**

SEND ADDRESS → ACCESS WITH * → CHANGE ORIGINAL

SLIDE 4 — Using Pointers to Modify Two Variables

Pointers can also be used to modify multiple variables.

Example:

None

```
#include <stdio.h>
```

```
void add(int *a, int *b)
```

```
{
```

```
    *a = *a + 10;
```

```
    *b = *b + 10;
```

```
}
```

```
int main()
{
    int x = 5;
    int y = 10;

    add(&x, &y);

    printf("%d %d", x, y);

    return 0;
}
```

Output:

None
15 20

Easy Analogy

Think of the function as someone given the **addresses of two boxes**.

None
x → 5 → *a
y → 10 → *b

The function can modify both boxes.

SLIDE 5 — RECAP

Pointers with Functions

Concept	Meaning
&x	Gets address of x

<code>int *p</code>	Receives an address
<code>*p</code>	Accesses the original value
<code>change(&x)</code>	Sends address to function
<code>*p = value</code>	Changes original variable

BIG MEMORY TRICK

SEND → RECEIVE → ACCESS → CHANGE

```
None
&x
↓
function(int *p)
↓
*p
↓
Original x changes
```

★ TAKEAWAY:

Pointers allow a function to **access and modify the original variable** instead of working only with a copy.

QUESTIONS

1 What does `&x` do when calling a function?

- A) Gets the value of `x`
- B) Gets the address of `x`
- C) Deletes `x`
- D) Changes `x` to 0

 **Hint:** `&` is used to get an address.

 Glossary:

Address → The location of a variable in memory.

2 What is the output?

None

```
void change(int *p)
{
    *p = 50;
}

int main()
{
    int x = 10;

    change(&x);

    printf("%d", x);
}
```

- A) 10
- B) 20
- C) 50
- D) Address of `x`

 **Hint:** The function receives the address of `x`.

 Glossary:

Original variable → The actual variable created in the calling function.

3 Complete the Code

None

```
int x = 10;

change(____);
```

Send the address of `x` to the function.

 **Hint:** Use the address operator.

 **Glossary:**

Address operator → `&`, used to get the address of a variable.

4 What does `*p = 100` do inside a function?

- A) Creates a new pointer
- B) Changes the original variable's value
- C) Deletes the original variable
- D) Changes the pointer to 100

 **Hint:** `*p` accesses the value at the stored address.

 **Glossary:**

Dereference → Accessing the value at the address stored in a pointer

5 Complete the Code

None

```
void change(int *p)
{
    _____ = 100;
}
```

Change the original variable to `100`.

 **Hint:** Use the dereference operator.

 **Glossary:**

Dereference operator → `*`, used to access the value pointed to by a pointer.